

Exception Handling - part 2

DAY 78
26/09/2025

What specific problem the Exception Handling trying to solve?

Suppose for Example I will try to open a file

I am trying to open file = open()

The file maybe locked
Not found
No permission
=====

} FAIL

Due to different situations the file opening will get failed, In this case the Operating System API's gives ERROR CODE

To Abstract the ERROR code given by Operating System API's ENUM'S are used

Enums:

t

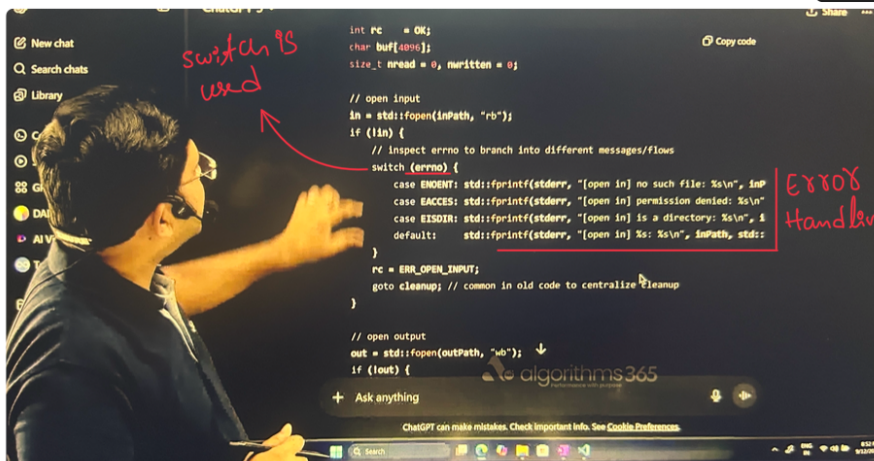
FileName: —

Security: —

// return codes instead of Exceptions

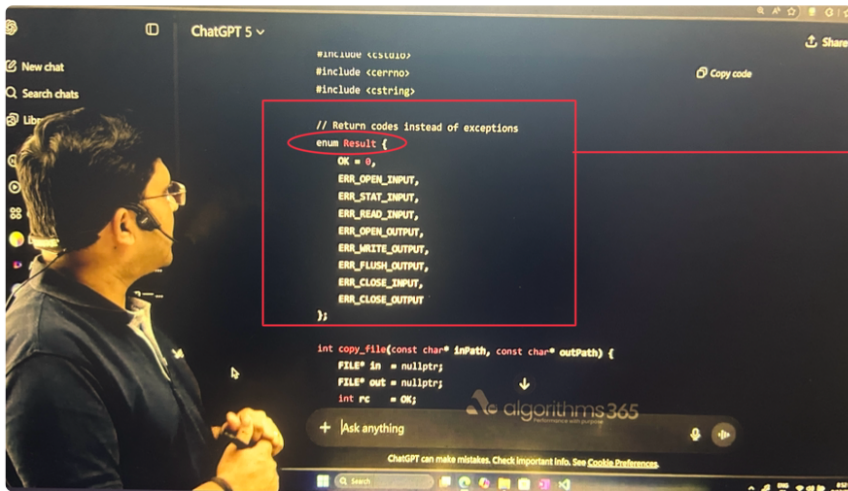
y

```
enum DivisionStatus {  
    SUCCESS,  
    DIVIDE_BY_ZERO,  
    INVALID_INPUT  
}
```



The this is the one of the dis advantage the code becomes messy and complex by using the switch statement, and code recompilation or updation is required if the codes want to add new error code

So this Enum used in switch statement and using the statuscode or error-code the exceptions are handled.



```
#include <stdio.h>
#include <errno.h>
#include <string.h>

// Return codes instead of exceptions
enum Result {
    OK = 0,
    ERR_OPEN_INPUT,
    ERR_STAT_INPUT,
    ERR_READ_INPUT,
    ERR_OPEN_OUTPUT,
    ERR_WRITE_OUTPUT,
    ERR_FLUSH_OUTPUT,
    ERR_CLOSE_OUTPUT,
};

int copy_file(const char* inPath, const char* outPath) {
    FILE* in = nullptr;
    FILE* out = nullptr;
    int rc = OK;
```

→ using Enums for Exception Handling created lot more complexity in the code.

- In Early programming languages (like C) Error handling was done By
 - Returning Error codes (E.g -1, NULL).
 - Setting global flags (errno in C)
- Problems With this
 - Easy to forget to check the return code
 - Mixed Happy Path logic with error checking, making code noisy
 - Resource leaks (files, memory) when a function failed But cleanup was skipped

Solution for this was structured Exception Handling

- Introduced in the 1980s - 1990s
 - Ada (1983), C++ (1990) Java (1995 with checked exceptions)
 - Python adapted it from its first release (1991)

Try - Catch - Finally Separated

- Happy Path logic → in the try
- Error recovery → in catch
- Cleanup always → in finally

Impact

Programs become cleaner, Safer, and more reliable

Happy Path :- Everything is working as expected in the code fine

In programming happy path means there are no errors everything working fine in the code

So the **try & try** block contains the happy path.

catch & try block contains logic of how the error is handled

finally & try block have the cleanup logic

So try, catch, finally are the 3 blocks of code used to handle exceptions smoothly

try &

_____ → Happy path
try

catch (Exception exceptionName) &

_____ → logic for Exception Handling
try

finally &

_____ → logic for cleanup
try

Some times during finally block code execution (during cleanup) it may also throw the exception so to handle that another try catch used in the finally block (nested try-catch)

So where the error is handled by nested try-catch, basically the error/exception is swallowed → this process also called swallowing.

finally {}

try {} == handling the exception or cleanup

```

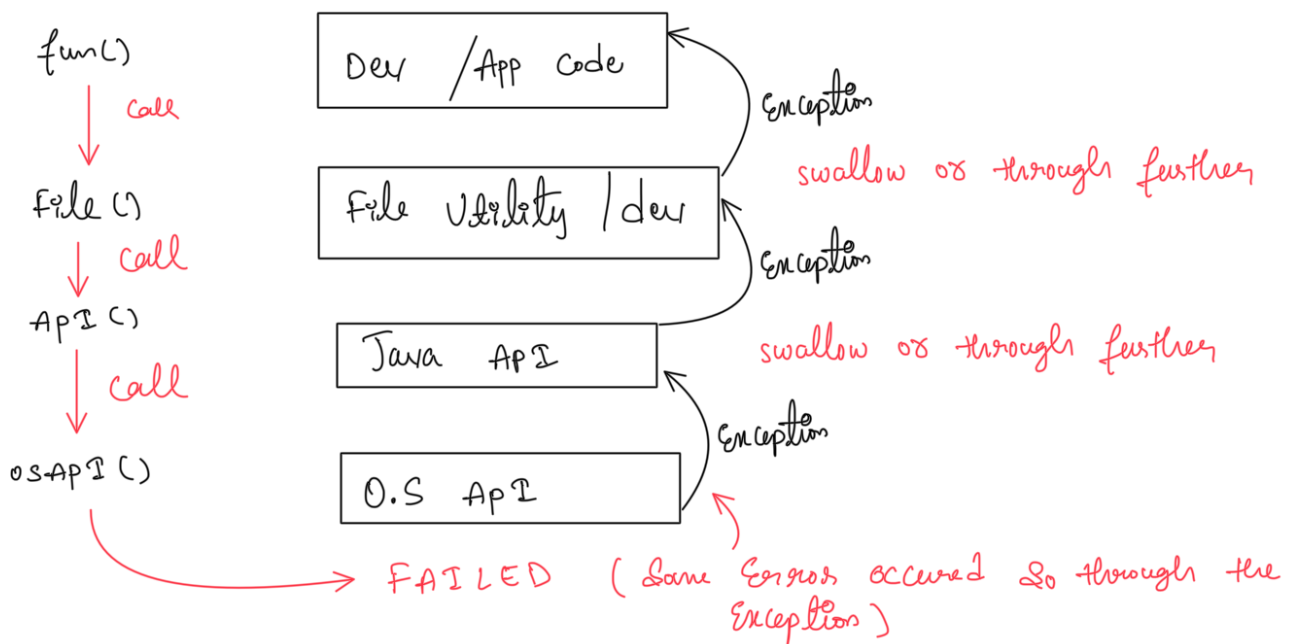
    }
    catch ( ) {}
    // swallow
    }

```

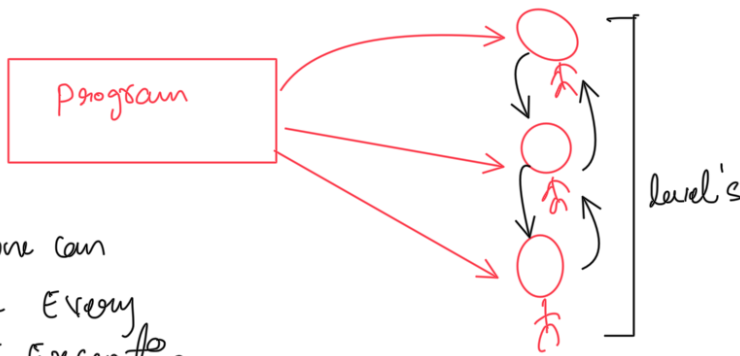
}

Basically the developer will not allow to take the exception further in the application, so it's swallowed

Every time the coder will decide whether the exception is further thrown to next level or it is swallowed at that level



NOTE: if exception is not handled by anyone then the application will get crashed and needs to be restarted



At particular level there is choice of handling Exception is throwing it to next level

Not Everyone can Handle the Every different Exception

PseudoCode Example

Try to Open the file
Read Contents

CATCH file-not-found
tell user "file missing"

FINALLY
close file if it was opened

```

import java.io.*;

public class TryCatchDemo {
    public static void readFile(String path) {
        FileReader reader = null;
        try {
            reader = new FileReader(path);
            BufferedReader br = new BufferedReader(reader);
            System.out.println("File contents: " + br.readLine());
        } catch (FileNotFoundException e) {
            System.out.println("Error: File not found at " + path);
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        } finally {
            System.out.println("Closing file (if opened)");
            try {
                if (reader != null) reader.close();
            } catch (IOException ignored) {}
        }
    }

    public static void main(String[] args) {
        readFile("ok.txt"); // normal case
        readFile("missing.txt"); // error case
    }
}
  
```

Users can also write code which throws Exception not only the code which handles the Exception.

using **throws** keyword

```

Example:

class AgeChecker {
    public static void checkAge(int age) throws Exception {
        if (age < 18) {
            throw new Exception("Age must be 18 or above!");
        }
        System.out.println("Valid age: " + age);
    }

    public static void main(String[] args) {
        try {
            checkAge(15); // This will throw exception
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
  
```

Tomorrow I will explore Exception Handling implementation part with some examples.